1. LSP确保扩展的“安全性”
OCP的核心是“扩展而非修改”，但盲目扩展可能引入错误（如子类破坏父类逻辑）。LSP通过约束子类的行为，确保子类可以安全替换父类，从而保证扩展的可靠性。
2. LSP消除“继承滥用”对OCP的破坏
若子类不遵守LSP（如重写父类方法时改变行为），会导致客户端代码依赖父类时出现不可预期的错误。此时，为修复问题可能需要修改父类或其他子类，违反OCP。LS
通过约束子类行为，避免此类问题。
3. LSP支持“面向接口编程”的OCP实践
OCP推荐“面向接口编程”（依赖抽象而非具体实现），而LSP确保接口的实现类（子类）可以无缝替换。通过定义稳定的接口（父类），并让子类遵守LSP，系统可以
活扩展新功能，同时保持现有代码的稳定性。
4. LSP降低“扩展”与“修改”的耦合
LSP要求子类与父类的行为兼容，因此扩展新功能时（新增子类），无需修改父类或其他子类的代码。这直接实现了OCP的“对扩展开放，对修改关闭”目标。

软件系统设计 2022 年考题整理

1. Please explain the Liskov Substitution Principle and how it contributes to the Open-Closed Principle.

请解释里氏替换原则以及它如何对开闭原则作出贡献。

2. Please explain how the Factory Method Pattern and Abstract Factory Pattern adhere to the Open-Closed Principle.

请解释工厂方法模式和抽象工厂模式如何遵循开闭原则。

3. What is the benefit of decoupling the Receiver from the Invoker in the Command Pattern?

在命令模式中，将接收者与调用者解耦的好处是什么？

4. There are two methods for propagating data to observers with the Observer design pattern: the push model and the pull model. Why would one model be preferable over the other? What are the trade-offs of each model?

在观察者设计模式中，有两种方法用于向观察者传播数据：推模型和拉模型。为什么一个模型比另一个更可取？每个模型的权衡是什么？

5. What is the difference between the categories of Creational Patterns and Structural Patterns? What categories do Prototype and Flyweight fall into, respectively? Explain why.

目的。创建对象；处理类或对象之间的组合关系。

创建型模式和结构型模式之间有什么区别？原型模式和享元模式分别属于哪个分类？请解释原因。

6. Describe the difference and relationship between software requirements, quality attributes, and architecturally significant requirements(ASR).

描述软件需求、质量属性和架构攸关的需求 (ASR) 之间的区别和关系。

7. Please define “Availability” as a quality attribute. What do MTBF and MTTR stand for? How to calculate the availability of a system(e.g.SLA) as the probability?

请给出质量属性中的“可用性的概念。MTBF 和 MTTR 分别代表什么？如何计算系统的可用性（例如 SLA）作为概率？

8. What are the generic design strategies applied in designing software? Give a concise working example with software architecture for these strategies.

设计软件时应用的通用设计策略有哪些？为这些策略提供一个带有软件架构的简明工作示例。

9. Why should a software architecture be documented using different views? Give the names and purposes of four example views.

为什么应该使用不同视图来记录软件架构？给出四个示例视图的名称和目的。

10. What are the risks, sensitivity points, and trade-off points associated with software architecture? Give an example for each of them.

对质量属性有负面影响/特定质量属性敏感/多个质量属性有影响 的架构决定。例子：数据库备份-性能，安全性；用户身份认证机制简单

软件架构涉及哪些风险、敏感点和权衡点？为每个点给出一个例子。

11. Design a Multi-instance pattern. Make sure that the numberof obiects of a class in the system cannot exceed a given constant n. Write code to implement a Multi-instance class.

设计一个多实例模式。确保系统中某个类的对象数量不会超过给定的常数 n。编写代码来实现一个多实例类。

12. Design an OA system to store the company departments and people in a tree structure. When a notice is issued,

you can set up all the people of the department or some individual people who need to be notified, and the notice will be displayed in the OA of those people. Drawthe design class diagram and write the code.

设计一个办公自动化系统，用于以树形结构存储公司部门和员工信息。当发布通知时，可以设置通知部门的所有员工或需要被通知的个别员工，通知将显示在这些人员的办公自动化系统中。绘制设计类图并编写代码。

13. Figure (a) below shows a distributed application with caches to improve its runtime performance. An online customer service system provides its clients access to their Bill Statement Information(BSI). The database stores BSI for clients. During a session, a client may require the BSI several times. In order to improve the performance, the BSI information for a particular client is cached. If the cache is loaded, then the BSI can be directly returned back to the client without access to the database. There are N servers clustered to serve the requests from clients. The server is load-balanced, a n deach time one server is scheduled for one client request. Al the caches on N servers need to synchronize their states. If the client request changes the states of one cache, there are two requirements:

REQ1. The changes are committed to the database;

REQ2. Al other cache states must be invalidated and they should reload the states from the database.

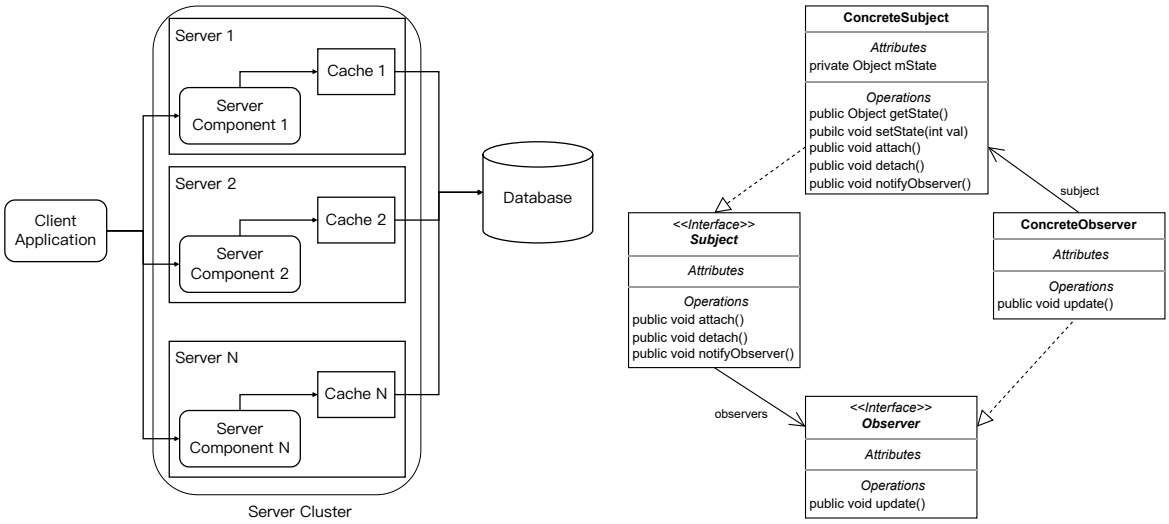
The current design in Figure (a) cannot meet the second requirement. Some important components are missing. Consider to apply architectural pattern(s) and tactic(s) to the design that can meet the second requirement, and complete the following questions:

(1) Identify extra component(s) that is(are) necessary for this design to meet second requirement. List each component name and its major functionalities.

(2) Map the roles of Observer pattern (such as subject, observer in Figure (b)) to whatever necessary components including your devised component(s) and the components in Figure (a).

(3) Draw sequence diagram to illustrate thecommunication between your devised components and original components in Figure (a).

(4) Assume that the servers are heterogeneous, and the protocols to access caches on different servers vary. For example, Server 1 supports Java RMI invocations to cache 1, Server 2 supports http JSP requests to cache 2, and server 3 supports SOAP requests to cache 3. It is further assumed that the cache interfaces are not consistent to each other due to the evolutionof system development and legacy problems. Explain how the concept of connectors or web services can be applied to provide a generic invocation to caches? You can choose either connector or web services for discussion.



(a) Distributed cache scenario (b) Observer design pattern

下图 (a) 显示了一个具有缓存的分布式应用程序，以提高其运行时性能。在线客户服务系统为客户提供访问其账单信息 (BSI) 的权限。数据库存储客户的 BSI。在一个会话期间，客户可能多次需要 BSI。为了提高性能，特定客户的 BSI 信息被缓存。如果缓存已加载，则可以直接将 BSI 返回给客户端，无需访问数据库。有 N 个服务器聚集在一起，为客户的请求提供服务。服务器进行负载平衡，每次一个服务器被调度为一个客户请求提供服务。所有 N 个服务器上的缓存需要同步它们的状态。如果客户的请求改变了一个缓存的状态，有两个要求：

REQ1. 更改必须提交到数据库；

REQ2. 所有其他缓存的状态必须失效，并且它们应该重新从数据库加载状态。

图 (a) 中的当前设计无法满足第二个要求。缺少一些重要的组件。考虑应用适合满足第二个要求的架构模式和策略，并回答以下问题：

(1) 确定满足第二个要求所需的额外组件。列出每个组件的名称和主要功能。

(2) 将观察者模式的角色（如图 (b) 中的主题、观察者）与必要的组件（包括您设计的组件）和图 (a) 中的组件进行映射。

(3) 绘制顺序图以说明您设计的组件与图 (a) 中原始组件之间的通信。

(4) 假设服务器是异构的，并且访问不同服务器上的缓存的协议各不相同。例如，服务器 1 支持对缓存 1 的 Java RMI 调用，服务器 2 支持对缓存 2 的 HTTP JSP 请求，服务器 3 支持对缓存 3 的 SOAP 请求。进一步假设由于系统开发的演进和遗留问题，缓存接口彼此之间不一致。请解释如何应用连接器或 Web 服务的概念来提供对缓存的通用调用？您可以选择连接器或 Web 服务进行讨论。